



Arquitectura de Computadoras

LABORATORIO 06

DESCRIPCIÓN DE SISTEMAS SECUENCIALES EN VHDL

MANUAL DE LAS SOLUCIONES A LOS EJERCICIOS DESARROLLADO POR: ADOLFO AYVAR SAAVEDRA

I. OBJETIVOS

Describir, simular y verificar circuitos lógicos secuenciales y máquinas de estado mediante código en VHDL y Quartus.

II. EJERCICIOS PROPUESTOS

PROBLEMA 01 (solo simulación)

Definir/asumir un patrón o secuencia de 7 bits.

Luego, desarrollar un sistema secuencial que detecte dicha secuencia en una entrada de datos binaria.

X = Entrada de bits, serial. En esta entrada puede aparecer el patrón o secuencia

Z = Salida, se pone en ALTO cuando la secuencia es detectada

ENTREGABLES

- Patrón a detectar:
- Diagrama de estados mostrando todos los estados, cambios de estado, entrada y salida. Dibujar en computadora o aplicación online.
- Captura de pantalla del código VHDL comentado.
- Captura de pantalla de resultados de simulación; deben mostrarse los estados, además de entrada y salida.
- Verificación de resultados, comentar y demostrar que cumple con la detección.

SOLUCION 01

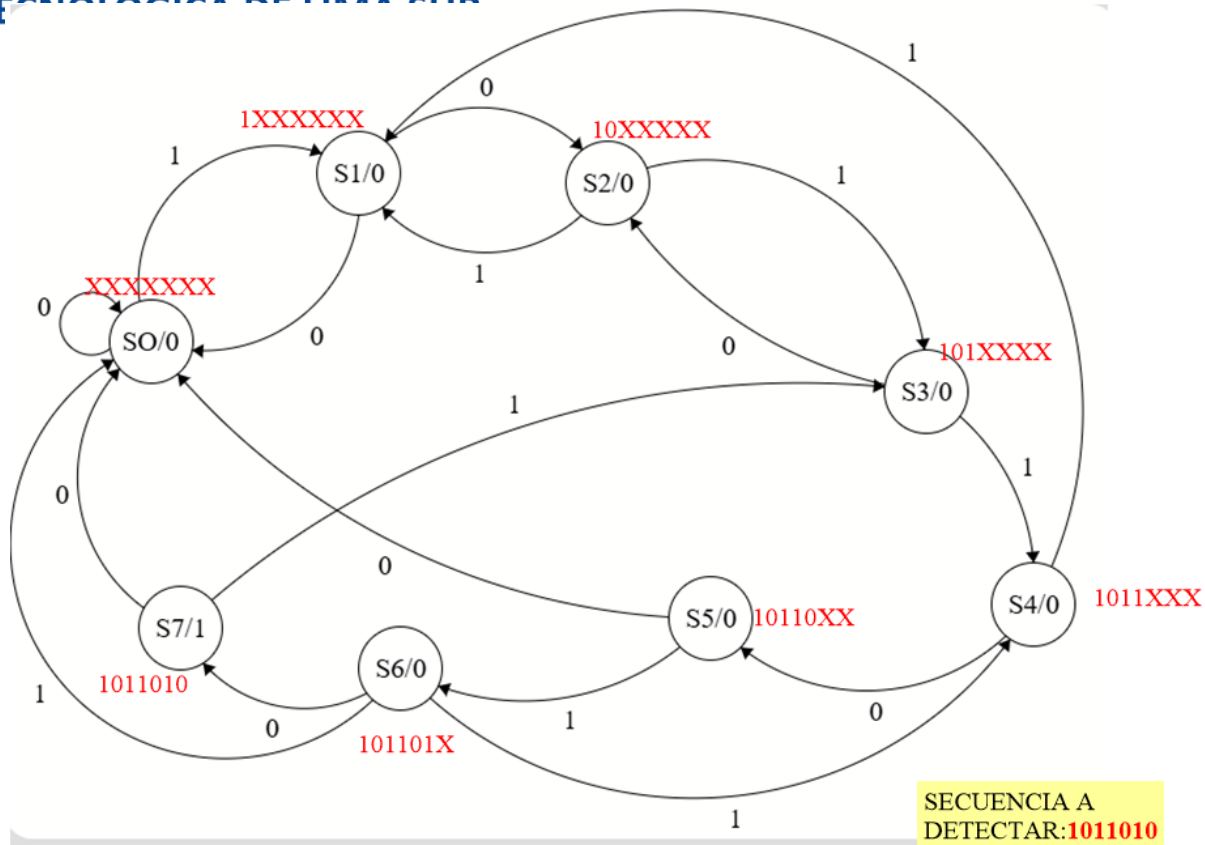
Contexto físico

El FPGA esta conectado a un sensor crítico, por ejemplo, un detector de sobrepresión en una caldera. Normalmente, la línea transmite datos de rutina. Sin embargo, si la presión sube a un nivel peligroso, el sensor inyecta la "Trama de Emergencia" (1011010). La FPGA está diseñada en hardware puro para que, al detectar esta trama, dispare la salida Z que abre una válvula de alivio en nanosegundos, sin esperar a que un procesador lea un código de software.

a)

Secuencia a detectar: 1011010 = 90 = Z (ASCII)

b) diagrama de Estados



c) Código

OBS: debe definirse una salida (llamada “debug_state”) para poder visualizar en el waveform el estado presente en el cual se encuentra la FSM

Cree un nuevo proyecto en QUARTUS PRIME con el mismo nombre que la entidad definida en este código **Detector_Secuencia**

Código VHDL-DETECTOR DE SECUENCIA

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Detector_Secuencia is
  Port (
    clk : in STD_LOGIC;
    reset : in STD_LOGIC;
    X : in STD_LOGIC;
    Z : out STD_LOGIC;
    debug_state : out std_logic_vector(2 downto 0)
  );
end Detector_Secuencia;

architecture Behavioral of Detector_Secuencia is
  -- Declaración de los estados para detectar 1011010
  type state_type is (S0, S1, S2, S3, S4, S5, S6, S7);
  signal current_state, next_state : state_type;
  -- Esta línea obliga a Quartus a mantener la señal visible:

begin

  -- Proceso secuencial: Actualización del estado actual con el reloj
  process(clk, reset)
  begin
    if reset = '1' then
      current_state <= S0; -- Retorno al estado inicial en reset
    elsif rising_edge(clk) then
      current_state <= next_state;
```

```

end if;
end process;

-- Proceso combinacional: Lógica de próximo estado y salida
process(current_state, X)
begin
    -- Valor por defecto de la salida
    Z <= '0';

    case current_state is
        when S0 =>
            if X = '1' then next_state <= S1; else next_state <= S0; end if;
        when S1 =>
            if X = '0' then next_state <= S2; else next_state <= S1; end if;
        when S2 =>
            if X = '1' then next_state <= S3; else next_state <= S0; end if;
        when S3 =>
            if X = '1' then next_state <= S4; else next_state <= S2; end if;
        when S4 =>
            if X = '0' then next_state <= S5; else next_state <= S1; end if;
        when S5 =>
            if X = '1' then next_state <= S6; else next_state <= S0; end if;
        when S6 =>
            if X = '0' then next_state <= S7; else next_state <= S4; end if;
        when S7 =>
            Z <= '1'; -- La salida se pone en ALTO al completar la secuencia
            -- Lógica para secuencias superpuestas (traslapadas)
            if X = '1' then next_state <= S3; else next_state <= S0; end if;
        when others =>
            next_state <= S0;
    end case;
end process;

process(current_state)
begin
    case current_state is
        when S0 => debug_state <= "000";
        when S1 => debug_state <= "001";
        when S2 => debug_state <= "010";
        when S3 => debug_state <= "011";
        when S4 => debug_state <= "100";
        when S5 => debug_state <= "101";
        when S6 => debug_state <= "110";
        when S7 => debug_state <= "111";
    end case;
end process;

end Behavioral;

```

Copia y ejecuta el código



File Edit View Project Assignments Processing Tools Window Help

SELECTOR_DE_OCTANAJE

Project Navigator Hierarchy

Entity: Instance

Cyclone IV E: EP4CE6E22C8

Detector_Secuencia

Tasks

Compilation

Task

- Compile Design
- Analysis & Synthesis
- Fitter (Place & Route)
- Assembler (Generate programming file)
- Timing Analysis
- EDA Netlist Writer
- Edit Settings
- Program Device (Open Programmer)

Detector_Secuencia.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Detector_Secuencia is
5     Port (
6         clk : in STD_LOGIC;
7         reset : in STD_LOGIC;
8         x : in STD_LOGIC;
9         z : out STD_LOGIC;
10        debug_state : out std_logic_vector(2 downto 0)
11    );
12 end Detector_Secuencia;
13
14 architecture Behavioral of Detector_Secuencia is
15     -- Declaración de los estados para detectar 1011010
16     type state_type is (S0, S1, S2, S3, S4, S5, S6, S7);
17     signal current_state, next_state : state_type;
18     -- Esta línea obliga a Quartus a mantener la señal visible:
19
20 begin
21
22     -- Proceso secuencial: Actualización del estado actual con el reloj
23     process(clk, reset)
24     begin
25         if reset = '1' then
26             current_state <= S0; -- Retorno al estado inicial en reset
27         elsif rising_edge(clk) then
28             current_state <= next_state;
29         end if;
30     end process;
31
32     -- Proceso combinacional: Lógica de próximo estado y salida
33     process(current_state, X)
34     begin
35         -- Valor por defecto de la salida
36         z <= '0';
37
38         case current_state is
39             when S0 =>
40                 if x = '1' then next_state <= S1; else next_state <= S0; end if;
41             when S1 =>
42                 if x = '0' then next_state <= S2; else next_state <= S1; end if;
43             when S2 =>
44                 if x = '1' then next_state <= S3; else next_state <= S0; end if;
45             when S3 =>
46                 if x = '1' then next_state <= S4; else next_state <= S2; end if;
47             when S4 =>
48                 if x = '0' then next_state <= S5; else next_state <= S1; end if;
49             when S5 =>
50                 if x = '1' then next_state <= S6; else next_state <= S0; end if;
51             when S6 =>
52                 if x = '0' then next_state <= S7; else next_state <= S4; end if;
53             when S7 =>
54                 z <= '1'; -- La salida se pone en ALTO al completar la secuencia
55         end case;
56     end process;
57 end;
```

Find... Find Next

Type	ID	Message
Information	293000	Quartus Prime Full Compilation was successful. 0 errors, 12 warnings

Simulacion: new->University Program UWF



UNIVERSIDAD NACIONAL TECNOLÓGICA DE LIMA SUR

Quartus Prime Lite Edition - C:/Users/HOME/Desktop/ARQUITECTURA DE COMPUTADORAS/LABS_QUARTUS/LAB_03/SELECTOR_DE_OCTANAJE - SELECTOR_DE_OCTANAJE

Entity/Instance

Cyclone IV E: EP4CE6E22C8

Detector_Secuencia

Tasks

Compilation

Task

Compile Design

Analysis & Synthesis

Fitter (Place & Route)

Assembler (Generate programming file)

Timing Analysis

EDA Netlist Writer

Edit Settings

Program Device (Open Programmer)

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Detector_Secuencia is
5 Port (
6     clk : in STD_LOGIC;
7     reset : in STD_LOGIC;
8     X : in STD_LOGIC;
9     Z : out STD_LOGIC;
10     debug_state : out std_logic_vector(2 downto 0)
11 );
12 end Detector_Secuencia;
13
14 architecture Behavioral of Detector_Secuencia is
15 -- Declaración de los estados para detectar 10110101
16 type state_type is (S0, S1, S2, S3, S4, S5, S6, S7);
17 signal current_state, next_state : state_type;
18 -- Esta línea obliga a Quartus a mantener la señal v
19
20 begin
21
22 -- Proceso secuencial: Actualización del estado actual
23 process(clk, reset)
24 begin
25     if reset = '1' then
26         current_state <= S0; -- Retorno al estado inicial
27     elsif rising_edge(clk) then
28         current_state <= next_state;
29     end if;
30 end process;
31
32 -- Proceso combinacional: Lógica de próximo estado y
33 process(current_state, X)
34 begin
35     -- valor por defecto de la salida
36     Z <= '0';
37
38     case current_state is
39         when S0 =>
40             if X = '1' then next_state <= S1; else next_state <= S0;
41         when S1 =>
42             if X = '0' then next_state <= S2; else next_state <= S1;
43         when S2 =>
44             if X = '1' then next_state <= S3; else next_state <= S2;
45         when S3 =>
46             if X = '1' then next_state <= S4; else next_state <= S3;
47         when S4 =>
```

New

State Machine File

SystemVerilog HDL File

Tcl Script File

Verilog HDL File

VHDL File

Memory Files

Hexadecimal (Intel-Format) File

Memory Initialization File

Verification/Debugging Files

In-System Sources and Probes File

Logic Analyzer Interface File

Signal Tap Logic Analyzer File

University Program VWF

Other Files

AHDL Include File

Block Symbol File

Chain Description File

Synopsys Design Constraints File

OK Cancel Help

Doble click donde indica la flecha y se abre la ventana, presionar Node_Finder

Simulation Waveform Editor - C:/Users/HOME/Desktop/ARQUITECTURA DE COMPUTADORAS/LABS_QUARTUS/LAB_03/SELECTOR_DE_OCTANAJE - SELECTOR_DE_OCTANAJE - [Waveform3.vwf]

File Edit View Simulation Help

Master Time Bar: 0 ps Pointer: 1.19 ns Interval: 1.19 ns Start:

Name Value at 0 ps

0 ps 40,0 ns 80,0 ns 120,0 ns 160,0 ns 200,0 ns 240,0 ns 280,0 ns 320,0 ns 360,0 ns 400,0 ns 440,0 ns 480,0 ns 520,0 ns 560,0 ns 600,0 ns 640,0 ns

Insert Node or Bus

Name: Use Node Finder to insert... OK

Type: INPUT Cancel

Value type: 9-Level

Radix: Binary

Bus width: 1

Start index: 0

Display gray code count as binary count

Node Finder...

En la Ventana emergente, presiona LIST y aparecerán la lista de entradas y salidas a usar

Node Finder

Named: * Filter: Pins: all OK

Look in: * ... List Cancel

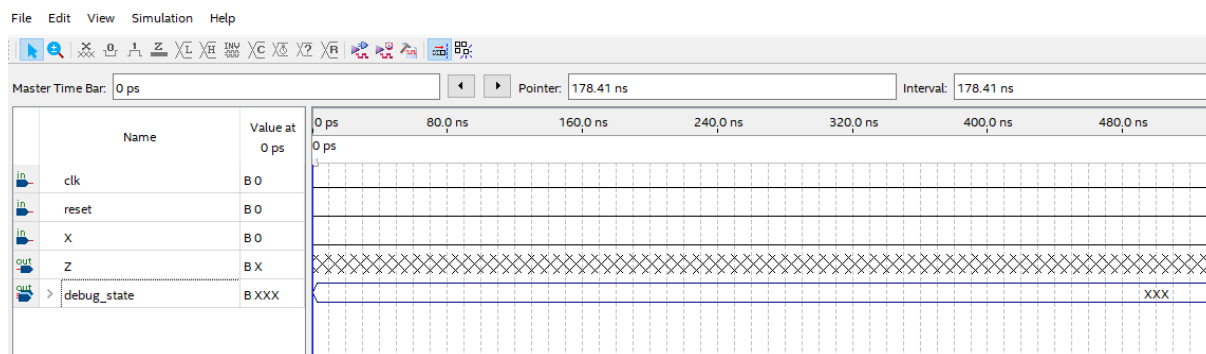
Nodes Found:

Name	Type
in clk	Input
out debug_state	Output Group
out debug_stat...	Output
out debug_stat...	Output
out debug_stat...	Output
in reset	Input
in X	Input
out Z	Output

Selected Nodes:

Name	Type
------	------

Darle forma adecuada a las entradas para probar la detección de la secuencia



OBS: no olvidar que antes de simular, en simulation settings debes eliminar novopot

Simulation Options

Caution: Improperly modifying these settings can cause the simulation to fail

HDL Language: ☐ Verilog ☒ VHDL (The language used for the testbench and netlist)

Functional Simulation Settings

Timing Simulation Settings

Testbench Generation Command (Functional Simulation):

```
quartus_eda --gen_testbench --tool=modelsim_oem --format=vhdl --write_settings_files=off Detector_Secuer
```

Netlist Generation Command (Functional Simulation):

```
quartus_eda --write_settings_files=off --simulation --functional=on --flatten_buses=off --tool=modelsim_oem
```

ModelSim Script (Functional Simulation):

```
onerror {exit -code 1}
vlib work
vcom -work work Detector_Secuencia.vho
vcom -work work Waveform.vwf.vht
vsim -novopt -c -t 1ps -L cyclonev -L altera -L altera_mf -L 220model -L sgate -L altera_insim work.Detector_S
vcd file -direction Detector_Secuencia.msim.vcd
vcd add -internal Detector_Secuencia_vhd_vec_tst/*
vcd add -internal Detector_Secuencia_vhd_vec_tst/i1/*
proc simTimestamp {} {
    echo "Simulation time: $::now ps"
    if {[string equal running [runStatus]]} {
        after 2500 simTimestamp
    }
}
after 2500 simTimestamp
run -all
quit -f
```

The command to generate the simulation netlist

Restore Defaults

Save

Cancel

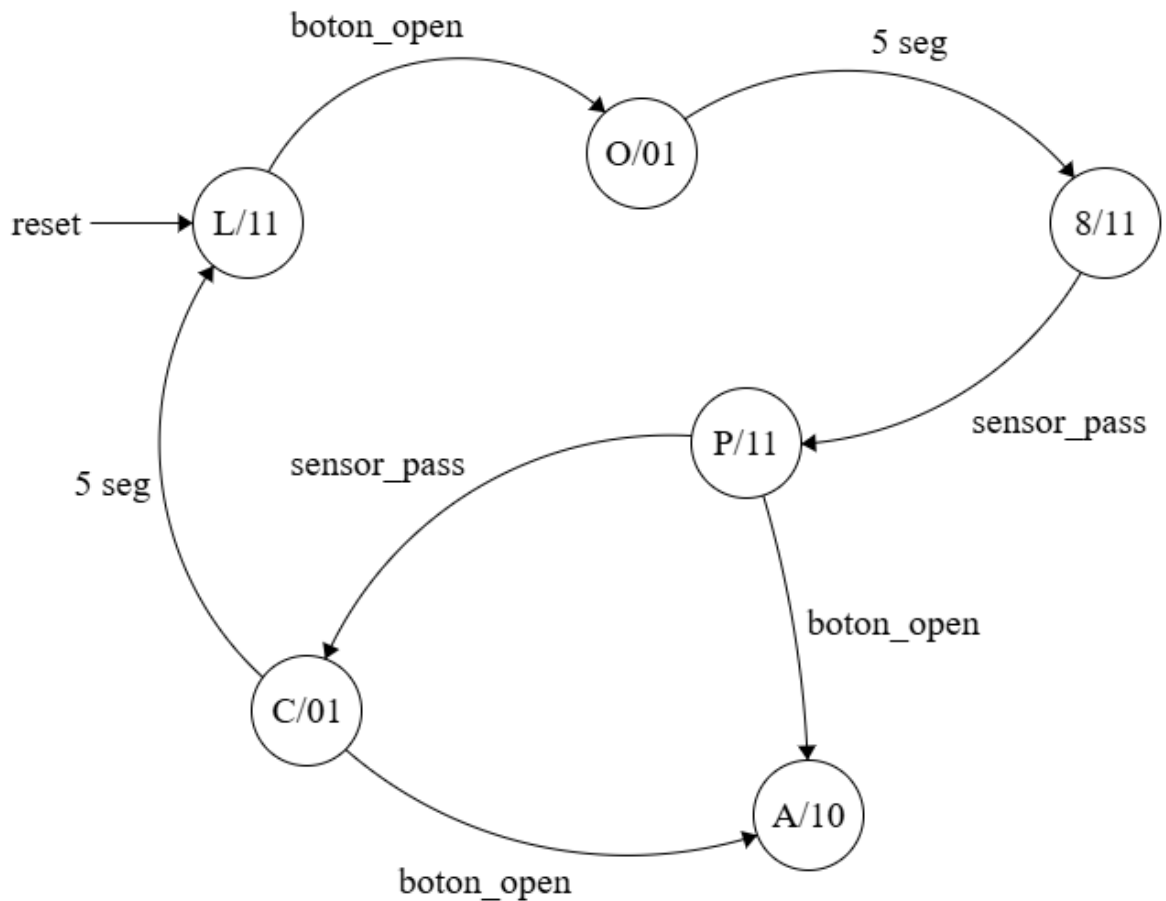
Resultado

[illegible]

Modificar/completar toda la tabla a criterio del grupo

Estado Actual	btn_open_falling	sensor_pass_falling	timer_count	Estado Siguiente	motor_open	led_alarm
LOCKED	1	X	X	OPENING	1	1
LOCKED	0	X	X	LOCKED	1	1
OPENING	X	1	X	ALARM_STATE	0	1
OPENING	X	0	< LIMIT-1	OPENING	0	1
OPENING	X	0	>= LIMIT-1	OPENED	0	1
OPENED	X	1	X	PASSING	1	1
OPENED	X	0	X	OPENED	1	1
PASSING	1	X	X	ALARM_STATE	1	1
PASSING	0	1	X	CLOSING	1	1
PASSING	0	0	X	PASSING	1	1
CLOSING	1	X	X	ALARM_STATE	0	1
CLOSING	0	X	< LIMIT-1	CLOSING	0	1
CLOSING	0	X	>= LIMIT-1	LOCKED	0	1
ALARM_STATE	X	X	X (reset=0)	LOCKED	1	0
ALARM_STATE	X	X	X (reset=1)	ALARM_STATE	1	0

- b) Dibujar el diagrama de estados mostrando todos los estados, cambios de estado, entrada y salidas.
Dibujar en computadora con aplicación **online**.



- c) Obtener/presentar el diagrama de estados empleado el asistente de Quartus para FSM.
- d) Mostrar las tablas de cambio de estados y la tabla de salidas generada con el asistente de Quartus.

	Source State	Destination State	Condition
1	ALARM_STATE	ALARM_STATE	(!reset_falling).(reset)
2	ALARM_STATE	LOCKED	(!reset_falling).(reset) + (reset_falling)
3	CLOSING	ALARM_STATE	(btn_open_falling).(reset)
4	CLOSING	CLOSING	(!timer_count[2]).(btn_open_falling).(reset)
5	CLOSING	LOCKED	(!timer_count[2]).(reset) + (timer_count[2]).(!btn_open_falling) + (timer_count[2]).(btn_open_falling).(reset)
6	LOCKED	LOCKED	(!btn_open_falling) + (btn_open_falling).(reset)
7	LOCKED	OPENING	(btn_open_falling).(reset)
8	OPENED	OPENED	(!sensor_pass_falling).(reset)
9	OPENED	PASSING	(sensor_pass_falling).(reset)
10	OPENED	LOCKED	(!reset)
11	OPENING	ALARM_STATE	(sensor_pass_falling).(reset)
12	OPENING	OPENED	(timer_count[2]).(!sensor_pass_falling).(reset)
13	OPENING	LOCKED	(!reset)
14	OPENING	OPENING	(!timer_count[2]).(!sensor_pass_falling).(reset)
15	PASSING	ALARM_STATE	(btn_open_falling).(reset)
16	PASSING	CLOSING	(!btn_open_falling).(sensor_pass_falling).(reset)
17	PASSING	PASSING	(!btn_open_falling).(sensor_pass_falling).(reset)
18	PASSING	LOCKED	(!reset)

- e) Mostrar captura de pantalla de resultados de simulación.

f) Realizar verificación de resultados, comentar y demostrar que cumple con las secuencias.

PRESENTACIÓN DE LABORATORIO

CARPETA ZIP CONTENIENDO:

- Informe en formato PDF con lo solicitado en cada pregunta.
- ZIP con las dos carpetas del proyecto realizado en Quartus.

2. Flujo de Trabajo en Quartus Prime

Para llevar este código a una Cyclone IV (por ejemplo, la popular tarjeta de desarrollo *EP4CE6* o *EP4CE22*), sigue estos pasos:

Paso A: Crear el Proyecto y Compilar

- Abre **Quartus Prime** (la versión *Lite Edition* es gratuita y soporta Cyclone IV).
- Ve a File -> New Project Wizard y crea un proyecto nuevo.
- Cuando el asistente te pida seleccionar el dispositivo (Device), busca la familia **Cyclone IV E** (o GX según tu tarjeta) y selecciona el modelo exacto de tu chip (ej. EP4CE6E22C8).
- Crea un archivo VHDL nuevo (File -> New -> VHDL File), pega el código anterior y guárdalo exactamente con el mismo nombre de la entidad: fsm_cyclone4FPGAV1.vhd.
- Ejecuta una **Compilación Inicial** haciendo clic en Processing -> Start Compilation para asegurarte de que no existan errores de sintaxis.

Paso B: Asignación de Pines (Pin Planner)

Para que la FPGA interactúe con el mundo real, debes conectar los puertos de tu código VHDL a los pines físicos del chip Cyclone IV conectado a los periféricos de tu placa (switches, LEDs, oscilador).

- Ve a Assignments -> Pin Planner.
- Verás una lista con tus puertos: clk, reset, btn_open, sensor_pass, motor_open, y led_alarm.
- En la columna **Location**, escribe el número de pin correspondiente según el manual de tu tarjeta de desarrollo.

Ejemplo típico en tarjetas Cyclone IV genéricas: (VER LA TABLA COMPLETA MAS ABAJO EN ESTE MANUAL)

- o clk -> PIN_23 (Reloj de 50 MHz de la placa)
 - o reset -> PIN_88 (Un switch o pulsador)
 - o btn_open -> PIN_89 (Otro switch)
 - o motor_open -> PIN_84 (Un LED indicador)
- En la columna **I/O Standard**, asegúrate de que estén configurados en 3.3-V LVTTL (el estándar por defecto para la mayoría de periféricos en estas placas).

Paso C: Generar el archivo de programación

- Cierra el Pin Planner.
- Vuelve a ejecutar la compilación completa (Start Compilation).
- Al finalizar con éxito, Quartus generará un archivo con extensión **.sof** (SRAM Object File) dentro de la carpeta output_files.

3. Cargar el diseño en la FPGA

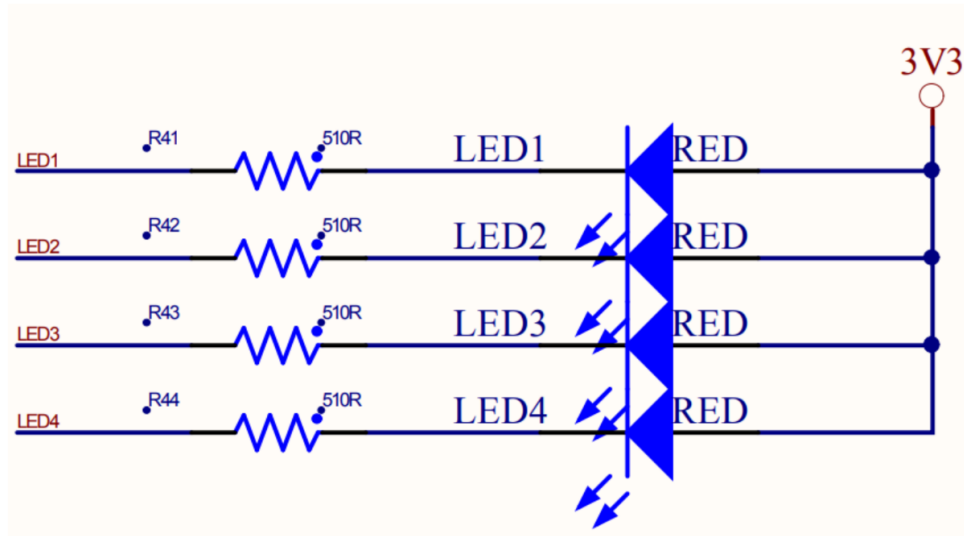
- Conecta tu tarjeta Cyclone IV a la computadora usando el programador **USB-Blaster**.
- En Quartus, abre la herramienta de grabación en Tools -> Programmer.
- Haz clic en **Hardware Setup** y asegúrate de seleccionar USB-Blaster.
- Asegúrate de que el modo esté en **JTAG**.
- Si el archivo .sof no aparece en la lista de forma automática, haz clic en **Add File...** y búscalo en la carpeta de salidas de tu proyecto.
- Marca la casilla **Program/Configure** al lado del nombre del archivo.
- Presiona **Start**.

Una vez que la barra llegue al 100%, la máquina de estados estará corriendo físicamente dentro del chip Cyclone IV. Podrás presionar los pulsadores asignados como entradas y ver cómo cambian los LEDs que definiste como salidas.

Banco de LEDs

- 4 LEDs
- Activos en bajo

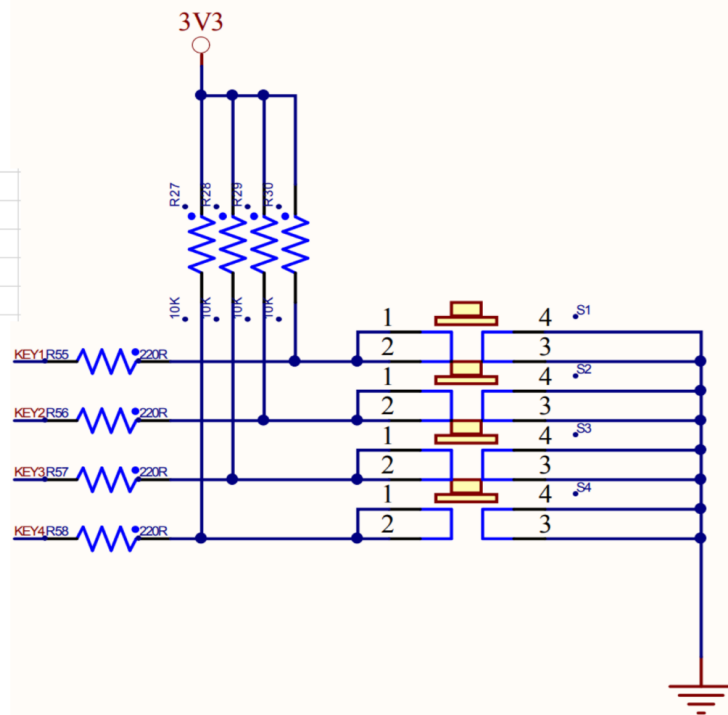
LED	Pin number
led1	84
led2	85
led3	86
led4	87



Banco de pulsadores

- 4 pulsadores
- Normalmente en alto

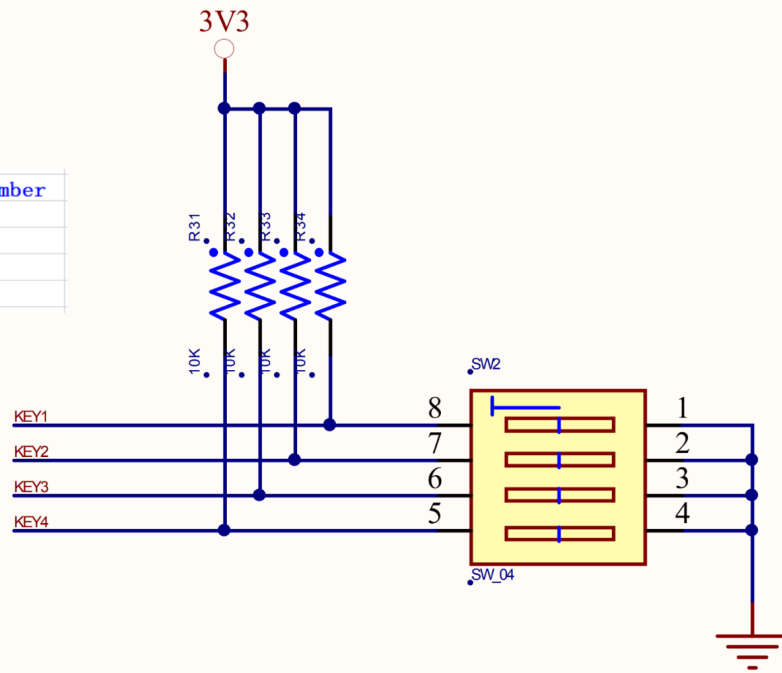
Keys	Pin number
KEY1	88
KEY2	89
KEY3	90
KEY4	91



DIPSWITCHES

- 4 switches

Dial switch	Pin number
ckey1	88
ckey2	89
ckey3	90
ckey4	91



SOLUCION AL EJERCICIO 2 (PARA IMPLEMENTACION)

Contexto de la FSM: un sistema de peaje

Esquema de la implementación

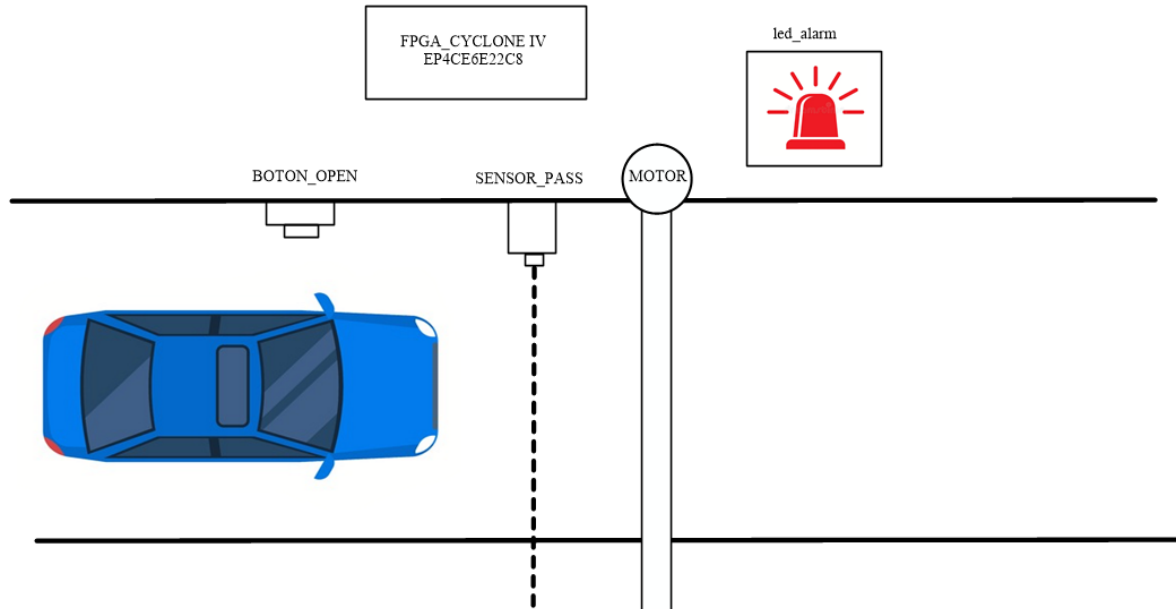
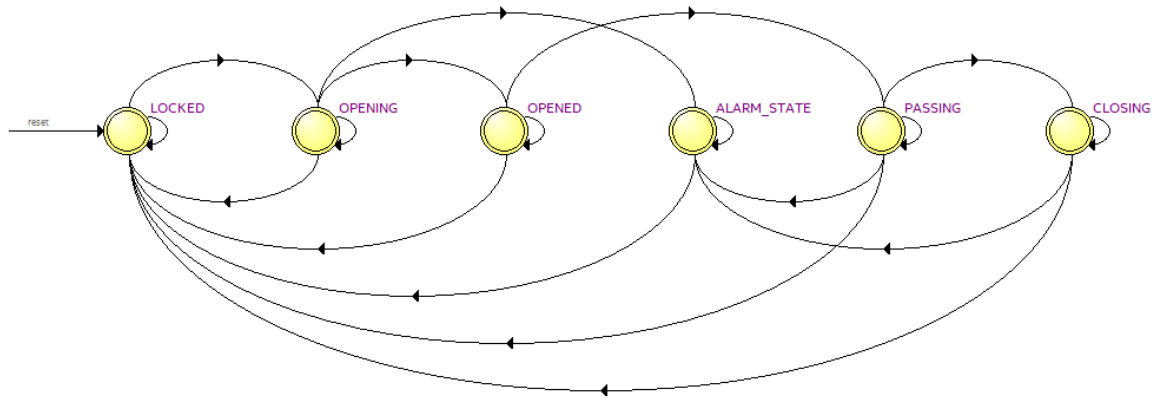


Diagrama de estados

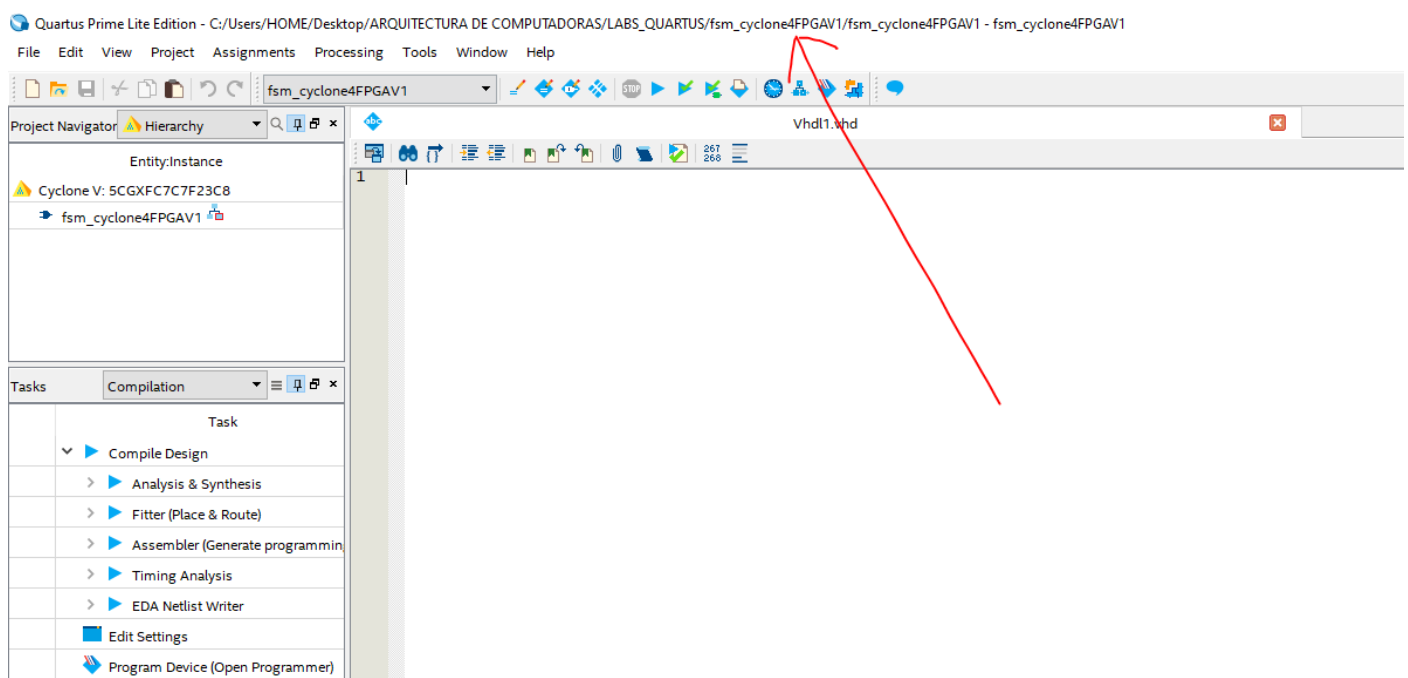


OBS: para que la simulación funcione en el software es necesario hacer algunas diferencias respecto de la implementación en hardware

- Ajustar el tiempo para que el contador trabaje solo 5 ciclos
- Eliminar los filtros anti rebote

OBS: puedes ejecutar este código en QUARTUS PRIME independientemente del código de implementación en físico, con el fin de obtener la waveform simulation

Creemos un nuevo proyecto en Quartus Prime con el mismo nombre de la entidad definida en el código VHDL



Pegue y ejecute el código

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity fsm_cyclone4FPGAV1 is
  Port (
    clk      : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    sensor_pass : in  STD_LOGIC;
    btn_open  : in  STD_LOGIC;
    motor_open : out STD_LOGIC;
    led_alarm  : out STD_LOGIC;
    segmentos : out STD_LOGIC_VECTOR(6 downto 0);
    digito_sel : out STD_LOGIC_VECTOR(3 downto 0);
    debug_state : out STD_LOGIC_VECTOR(6 downto 0)
  );
end fsm_cyclone4FPGAV1;

architecture RTL of fsm_cyclone4FPGAV1 is
  type T_state is (LOCKED, OPENING, OPENED, PASSING, CLOSING, ALARM_STATE);
  signal state : T_state := LOCKED;

  -- Usamos las entradas directamente para la simulación
  signal btn_open_reg, btn_open_falling : STD_LOGIC := '0';
  signal sensor_pass_reg, sensor_pass_falling : STD_LOGIC := '0';
  signal reset_reg, reset_falling : STD_LOGIC := '0';

  constant TIMER_5S_LIMIT : integer := 5; -- Valor muy pequeño para ver la transición
  signal timer_count : integer range 0 to TIMER_5S_LIMIT := 0;

begin
  digito_sel <= "0000";

  process(clk)
  begin
    if rising_edge(clk) then
      -- 1. DETECTOR DE FLANCO DE BAJADA (DIRECTO)
      btn_open_reg    <= btn_open;
      sensor_pass_reg  <= sensor_pass;
      reset_reg        <= reset;

      btn_open_falling <= (not btn_open) and btn_open_reg;
      sensor_pass_falling <= (not sensor_pass) and sensor_pass_reg;
      reset_falling    <= (not reset) and reset_reg;

      -- 2. FSM
      motor_open <= '1';
      led_alarm  <= '1';

      if reset = '0' then -- Reset asíncrono para simulación más fácil
        state <= LOCKED;
        timer_count <= 0;
      else
        case state is
          when LOCKED =>
            if btn_open_falling = '1' then state <= OPENING; end if;

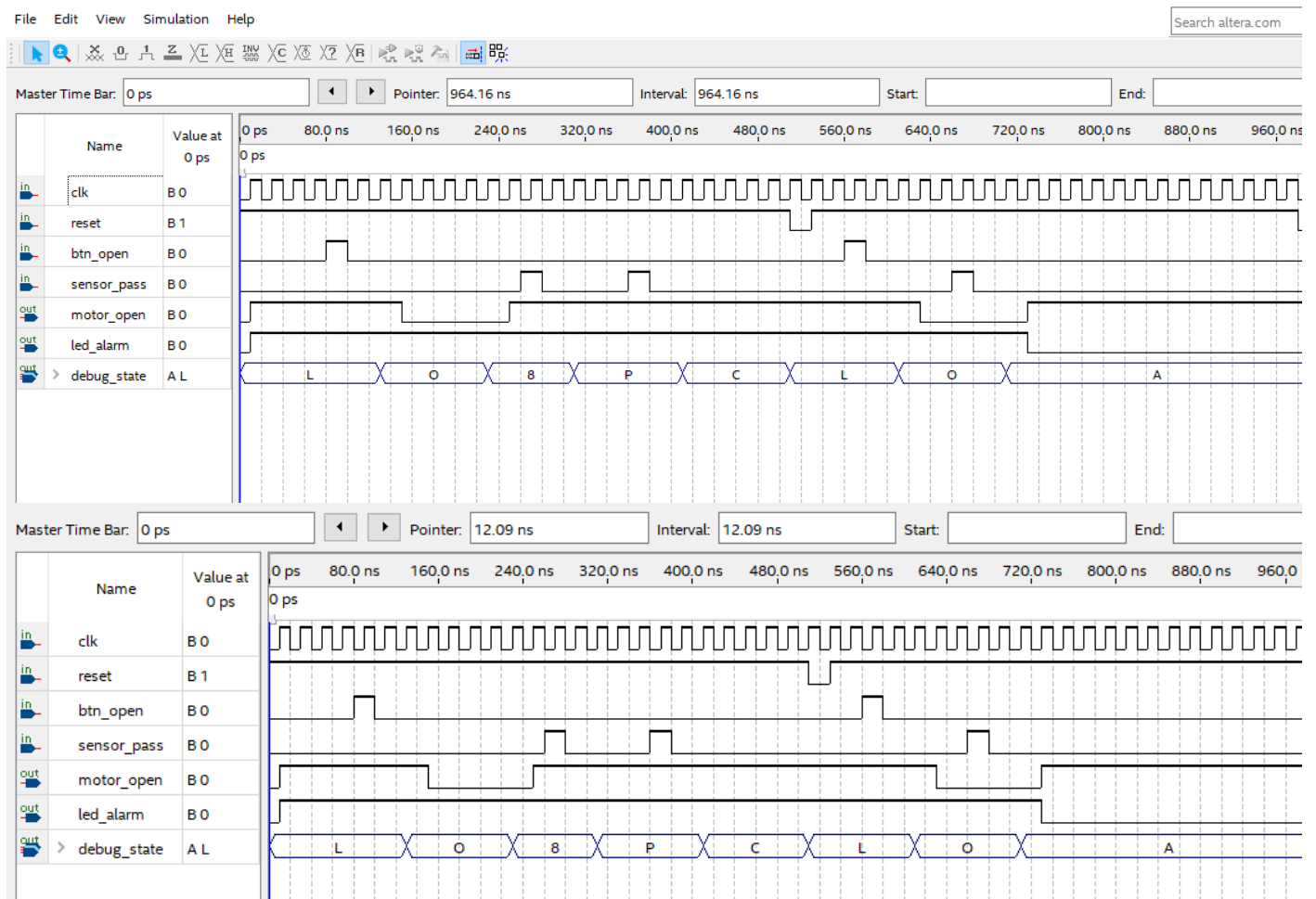
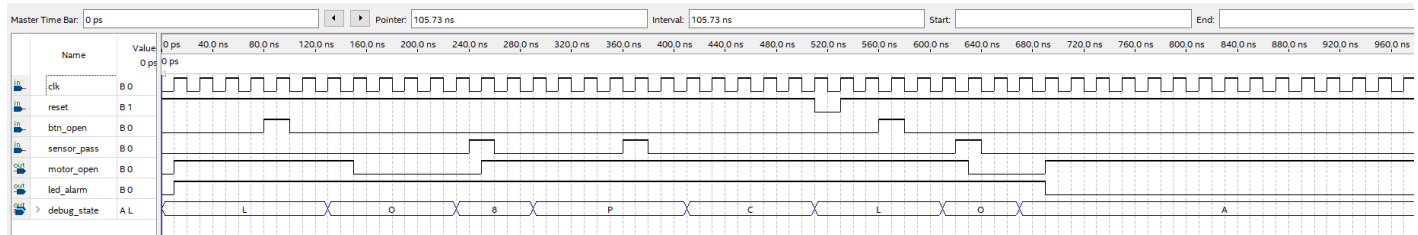
          when OPENING =>
            motor_open <= '0';
            if sensor_pass_falling = '1' then state <= ALARM_STATE;
            elsif timer_count >= (TIMER_5S_LIMIT - 1) then state <= OPENED; timer_count <= 0;
            else timer_count <= timer_count + 1; end if;
          -- ... (other states would follow here)
        end case;
      end if;
    end if;
  end process;
end;
```

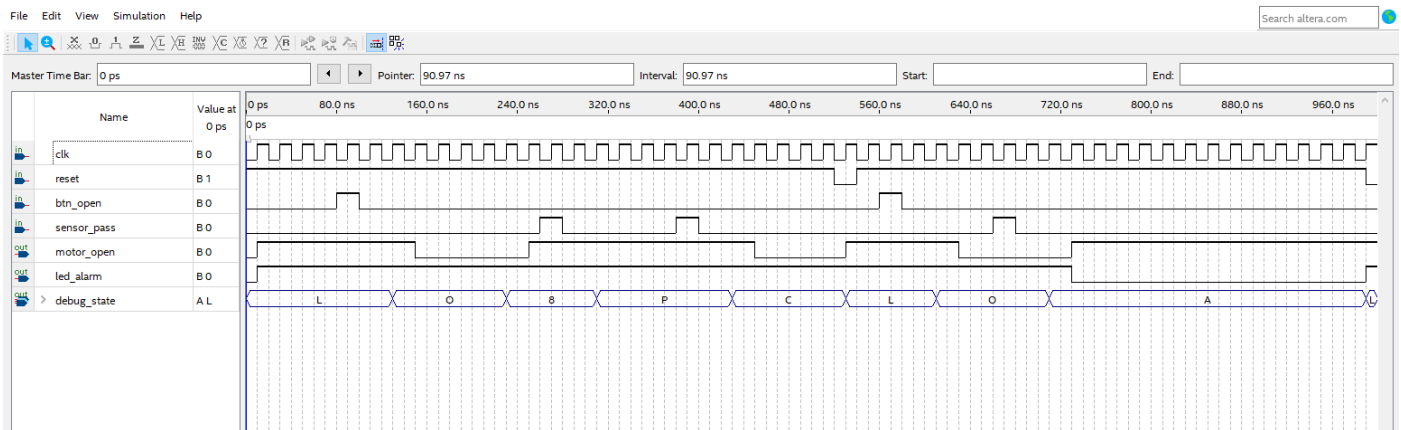
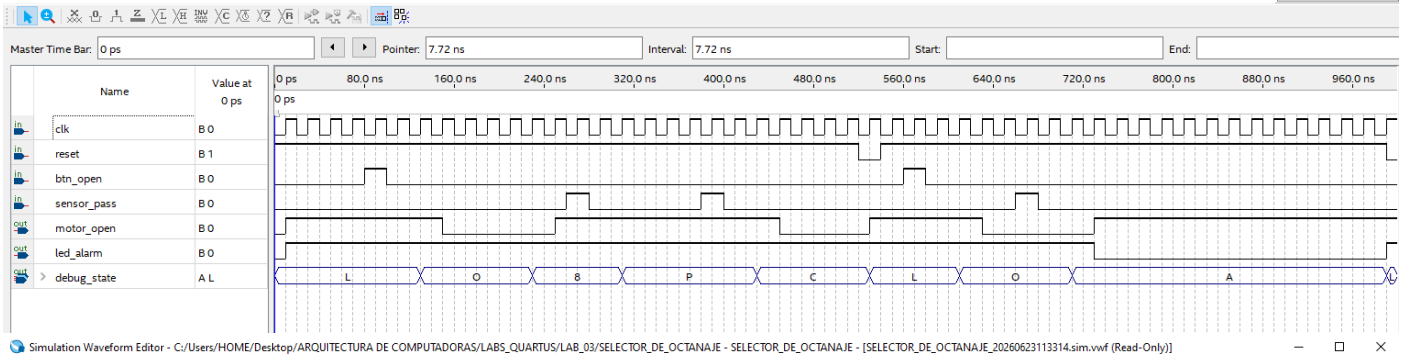

VERSION_PARA_SIMULACION

Replique los estímulos de acuerdo a la imagen, la simulación arrojará resultados representando la FSM exitosa

Clock con un periodo de 20ns

Debug_state en Radix->ASCII





CIRCUITO VALIDADO QUE FUNCIONA EN FISICO

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity fsm_cyclone4FPGAV1 is

Port (

-- Entradas globales y de control

clk : in STD_LOGIC; -- 50 MHz

reset : in STD_LOGIC; -- KEY1

sensor_pass : in STD_LOGIC; -- KEY2

btn_open : in STD_LOGIC; -- KEY3

-- Salidas físicas (Activas en Bajo)

motor_open : out STD_LOGIC;

led_alarm : out STD_LOGIC;

-- Para el display de 7 segmentos (Activo en Bajo)

segmentos : out STD_LOGIC_VECTOR(6 downto 0);

digito_sel : out STD_LOGIC_VECTOR(3 downto 0);

-- Variable para depuracion en el visualizador de WAVEFORM

debug_state : out STD_LOGIC_VECTOR(6 downto 0)

);

end fsm_cyclone4FPGAV1;

architecture RTL of fsm_cyclone4FPGAV1 is

type T_state is (LOCKED, OPENING, OPENED, PASSING, CLOSING, ALARM_STATE);

signal state : T_state := LOCKED;

-- Señales limpias después del filtro de rebotes

signal btn_open_clean : STD_LOGIC := '1';

signal sensor_pass_clean : STD_LOGIC := '1';

signal reset_clean : STD_LOGIC := '1';

-- Registros para detectar el flanco de bajada limpio

signal btn_open_reg : STD_LOGIC := '1';

signal btn_open_falling : STD_LOGIC := '0';

signal sensor_pass_reg : STD_LOGIC := '1';

signal sensor_pass_falling : STD_LOGIC := '0';

```
signal reset_reg      : STD_LOGIC := '1';  
signal reset_falling  : STD_LOGIC := '0';
```

```
-- Constante para el filtro antirebote (40 ms)  
constant DEBOUNCE_LIMIT : integer := 2000000;
```

```
-- Contadores independientes para el antirebote  
signal count_open  : integer range 0 to DEBOUNCE_LIMIT := 0;  
signal count_sensor : integer range 0 to DEBOUNCE_LIMIT := 0;  
signal count_reset : integer range 0 to DEBOUNCE_LIMIT := 0;
```

```
-----  
-- NUEVAS CONSTANTES Y SEÑALES PARA EL TEMPORIZADOR DE 5 SEGUNDOS  
-----
```

```
-- 50 MHz * 5 segundos = 250,000,000 ciclos de reloj
```

```
constant TIMER_5S_LIMIT : integer := 250000000;  
--constant TIMER_5S_LIMIT : integer := 10;  
signal timer_count      : integer range 0 to TIMER_5S_LIMIT := 0;
```

```
begin
```

```
-- Activar los 4 dígitos del bloque (Ánodo común = '0' enciende)  
digito_sel <= "0000";
```

```
-- Proceso secuencial síncrono
```

```
process(clk)
```

```
begin
```

```
    if rising_edge(clk) then
```

```
        -----  
        -- 1. FILTROS ANTIREBOTE (DEBOUNCERS) POR HARDWARE  
        -----
```

```
        if btn_open = '0' then  
            if count_open < DEBOUNCE_LIMIT then  
                count_open <= count_open + 1;  
            else  
                btn_open_clean <= '0';  
            end if;  
        else  
            count_open <= 0;  
            btn_open_clean <= '1';  
        end if;
```

```
        if sensor_pass = '0' then  
            if count_sensor < DEBOUNCE_LIMIT then  
                count_sensor <= count_sensor + 1;  
            else  
                sensor_pass_clean <= '0';  
            end if;  
        else  
            count_sensor <= 0;  
            sensor_pass_clean <= '1';  
        end if;
```

```
        if reset = '0' then  
            if count_reset < DEBOUNCE_LIMIT then  
                count_reset <= count_reset + 1;  
            else  
                reset_clean <= '0';  
            end if;  
        else  
            count_reset <= 0;  
            reset_clean <= '1';  
        end if;
```

```
        -----  
        -- 2. DETECTOR DE FLANCO DE BAJADA  
        -----
```

```
        btn_open_reg      <= btn_open_clean;  
        sensor_pass_reg   <= sensor_pass_clean;  
        reset_reg         <= reset_clean;
```

```
btn_open_falling <= (not btn_open_clean) and btn_open_reg;  
sensor_pass_falling <= (not sensor_pass_clean) and sensor_pass_reg;  
reset_falling <= (not reset_clean) and reset_reg;
```

-- 3. MÁQUINA DE ESTADOS FINITOS (FSM) CON TEMPORIZACIÓN

-- Valores por defecto (Lógica inversa: '1' apaga/desactiva)

```
motor_open <= '1';  
led_alarm <= '1';
```

```
if reset_falling = '1' then  
  state <= LOCKED;  
  timer_count <= 0; -- Reiniciar temporizador en reset  
else  
  case state is  
  
    when LOCKED =>  
      timer_count <= 0; -- Mantener el contador en 0 mientras no se use  
      if btn_open_falling = '1' then  
        state <= OPENING;  
      end if;  
  
    when OPENING =>  
      motor_open <= '0'; -- Motor encendido (M=0)  
  
      -- Prioridad 1: Interrupción por Sensor (Alarma)  
      if sensor_pass_falling = '1' then  
        state <= ALARM_STATE;  
        timer_count <= 0;  
      -- Prioridad 2: Control del tiempo (5 segundos)  
      elsif timer_count >= (TIMER_5S_LIMIT - 1) then  
        state <= OPENED;  
        timer_count <= 0; -- Resetear para la siguiente etapa  
      else  
        timer_count <= timer_count + 1; -- Incrementar tiempo  
      end if;  
  
    when OPENED =>  
      timer_count <= 0;  
      motor_open <= '1'; -- Motor apagado (M=1)  
      if sensor_pass_falling = '1' then  
        state <= PASSING;  
      end if;  
  
    when PASSING =>  
      timer_count <= 0;  
      -- Prioridad 1: Interrupción por botón (Alarma)  
      if btn_open_falling = '1' then  
        state <= ALARM_STATE;  
      -- Prioridad 2: El auto termina de pasar  
      elsif sensor_pass_falling = '1' then  
        state <= CLOSING;  
      end if;  
  
    when CLOSING =>  
      motor_open <= '0'; -- Motor encendido (M=0)  
      -- Prioridad 1: Interrupción por botón (Alarma)  
      if btn_open_falling = '1' then  
        state <= ALARM_STATE;  
        timer_count <= 0;  
      -- Prioridad 2: Control del tiempo (5 segundos)  
      elsif timer_count >= (TIMER_5S_LIMIT - 1) then  
        state <= LOCKED;  
        timer_count <= 0;  
      else  
        timer_count <= timer_count + 1;  
      end if;  
  
    when ALARM_STATE =>  
      timer_count <= 0;  
      led_alarm <= '0'; -- Bocina/Alarma encendida
```

```

        if reset_falling = '1' then
            state <= LOCKED;
        end if;

        when others =>
            state <= LOCKED;
        end case;
    end if;
end if;
end process;

```

-- 4. DECODIFICADOR DEL DISPLAY (Ánodo Común: '0'=Prende)

```

process(state)
begin
    case state is
        when LOCKED =>
            segmentos <= "1000111"; -- Letra "L"

        when OPENING =>
            segmentos <= "1000000"; -- Letra "O"

        when OPENED =>
            segmentos <= "0000000"; -- Letra "8"

        when PASSING =>
            segmentos <= "0001100"; -- Letra "P"

        when CLOSING =>
            segmentos <= "1000110"; -- Letra "C"

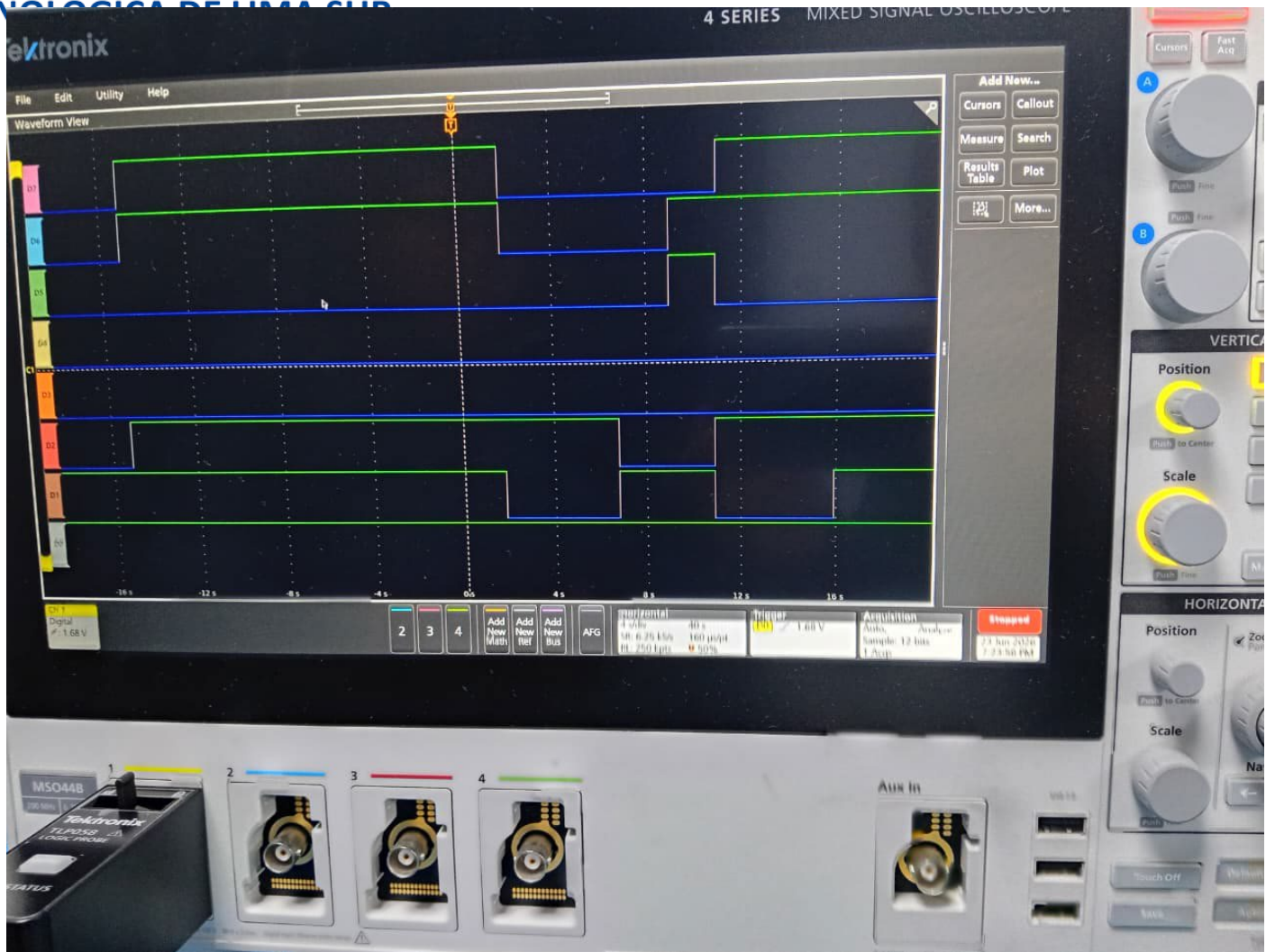
        when ALARM_STATE =>
            segmentos <= "0001000"; -- Letra "A"

        when others =>
            segmentos <= "1111111"; -- Todo apagado
    end case;
end process;

-- Proceso para visualizar el estado en la simulación
process(state)
begin
    case state is
        when LOCKED    => debug_state <= "1001100"; -- 76 (L)
        when OPENING   => debug_state <= "1001111"; -- 79 (O)
        when OPENED     => debug_state <= "0111000"; -- 56 (8)
        when PASSING    => debug_state <= "1010000"; -- 80 (P)
        when CLOSING    => debug_state <= "1000011"; -- 67 (C)
        when ALARM_STATE => debug_state <= "1000001"; -- 65 (A)
        when others     => debug_state <= "0000000";
    end case;
end process;
end RTL;

```





Evidencias

<https://youtu.be/t3R79dXACjQ>

OBSERVAR LOS ESTADOS DE LA FSM EN LOS DISPLAY DEL FPGA COMO SALIDA DE DEPURACION/CORROBORACION DEL CORRECTO FUNCIONAMIENTO DEL SISTEMA

